Progress Report — Stock Return Prediction with Neural Networks

Date: 2026-08-08

1. Objective

Following the project brief, the goal is to design a database of historical NASDAQ (and NYSE) daily OHLCV data, define input/output features for a neural network, and train a model — shared across all tickers — to predict short-horizon stock returns. Two complementary architectures were outlined in the brief: a feed-forward network on moving-average/slope features, and an LSTM that consumes a rolling window of daily history.

Work so far has focused on building the data pipeline and a from-scratch LSTM implementation, per the second instruction set.

2. Data pipeline

- **Source**: Yahoo Finance, daily OHLCV, full history per ticker.
- **Bug fix — price adjustment**: the initial download used `auto_adjust=False`, which left Open prices unadjusted relative to Close (e.g., after splits/dividends), producing inconsistent intraday/overnight return calculations. Corrected to adjust all price columns consistently.
- **Column normalization**: standardized column names across tickers so downstream code can treat every ticker uniformly.
- **Ordering bug**: data was originally sorted by `(Date, Ticker)`. This broke rolling-window feature computation (moving averages, slopes, returns), since windows would mix rows across different tickers. Fixed by sorting by `(Ticker, Date)` so each ticker's time series is contiguous before windowing.
- **Feature decisions**:
- Dropped daily return as an explicit feature, since it is fully determined by intraday return × overnight return (redundant, not independent information).
- Added z-score normalization for Volume, which otherwise varies by orders of magnitude across tickers.
- **Diagnostics**: added plotting utilities to visually sanity-check the fetched and processed series before feeding them to the model.

3. Model architecture

Implemented a custom LSTM from scratch in PyTorch (`model/model.py`), rather than using `nn.LSTM`, for transparency and control over initialization:

- `FEBCellLSTM`: a single LSTM cell with explicit input, forget, candidate, and output gates.
- Gate weights use Xavier uniform initialization.
- Forget-gate bias is initialized to `1` (`sigmoid(1) ≈ 0.73`), a standard trick that biases the network toward retaining prior memory early in training and improves gradient flow.
- `FEBLSTM`: wraps the cell, unrolls it over a window of `number_of_days` time steps, and maps the final hidden state to the output through a linear layer.
- The network is intended to be shared across all tickers (one global model, not one per stock), consistent with the brief.

4. Training and evaluation

Implemented in `model/training.py` and `model/inference.py`:

- Loss/metrics tracked: MAE, RMSE, MAPE, and R^2.
- Experiment tracking via Weights & Biases.
- Iterative engineering fixes along the way:
- Corrected the order of gradient clipping relative to `optimizer.step()`.
- Tuned learning rate and settled on a batch size of 128.
- Added `torch.compile` with Triton, pinned-memory data loading, and per-epoch logging.
- Checkpointing keeps only the best model by validation loss.
- **Debugging train/val discrepancy**: a persistent, large gap between training and validation loss was traced back to the ticker/date sorting bug described above — once fixed, the discrepancy was resolved.

5. Current status

- The data pipeline (fetch → clean → normalize → window) and the LSTM training/inference pipeline are functional, with checkpoints saved from completed training runs.
- Per the most recent commit, the next planned step is to build a statistical volatility model (GARCH family) to characterize volatility structure independently of the neural network, likely as a benchmark/complement to the LSTM — in line with the brief's request to compare architectures via a mean-error / error-std-dev Pareto frontier.